Campus Resource Reservation System: A UI-Independent Reservation Module and Desktop Application

Department of Computer Engineering, Kocaeli University, Software Laboratory I, Fall 2026-2027, Project I

Abstract

***This report describes the Campus** Resource Reservation System developed for Software Laboratory I. The system shares scarce campus resources (meeting rooms, laboratories, 3D printers and a GPU server) among students, researchers and technicians under a Resource Point (KP) economy. Its core is a reservation module that is independent of any user interface: it reads an initial configuration file, executes a JSON Lines command script in order, and writes exactly one result line per command. The module implements thirteen ordered validation rules, maintenance windows, preparation times, a consecutive-use limit, occupancy-based dynamic pricing with a peak surcharge, a weekly quota, all-or-nothing package reservations, a reservation state machine, a reliability score, and an automatic waiting list. A deterministic, event-driven simulated clock applies simultaneous events in a fixed order. All operations pass through one engine entry point, so the command-line path and the desktop application use the same rules. A data-driven test suite checks each command family and the interactions between the clock, the waiting list and the state machine.*

Index Terms — resource reservation, dynamic pricing, state machine, waiting list, discrete-event simulation, Python.

1. Introduction

Motivation. A newly established university has only a few meeting rooms, laboratories, 3D printers and one GPU server. Under a first-come-first-served policy, a few users can block popular time slots, and users who book a slot but never arrive waste capacity. The

university therefore defined its own sharing rules: payment with weekly-renewed Resource Points, prices that rise with demand, usage rules that depend on the resource type (a printer must cool down and a laboratory must be ventilated between two uses), and a reliability score that rewards users who respect their reservations.

Problem statement. We must implement these rules in a reservation module that works independently of the user interface, and a desktop application that drives the same module. Hidden test scenarios compare the result lines field by field, so the implementation must be exact (rounding, boundaries, error-code priority, event order) and deterministic. All parameters (resource types, roles, quotas, resources, maintenance windows, users and the initial clock) come from the start file and are not hard-coded.

Overview. The module is written in Python 3.11 with Pydantic v2 models. It provides (i) thirteen ordered checks for a reservation request, (ii) pricing in 15-minute slices based on band occupancy, (iii) atomic packages of two to four items, (iv) the states ONAYLI, KULLANIMDA, TAMAMLANDI, IPTAL and GELMEDI with refunds and reliability changes, (v) a reliability-ordered waiting list with automatic assignment, and (vi) a simulated clock that processes weekly resets, completions, no-shows and automatic assignments chronologically. Section 2 explains the method, Section 3 presents the tests, Section 4 justifies the design decisions, and Section 5 concludes.

2. Methodology

2.1. System Architecture

The code base is one Python package, `yazilim_lab_project`, with four parts (Fig. 1).

- `models/` contains the typed data structures: the initial data, reservations, waiting entries, application state, commands, results and events.
- `rules/` contains validation only. Small predicates are in `rules/atomic/`. Each command has an `all_rules_*` function that runs the predicates in the specified order and returns `Ok(data)` or `Err(code)`. Rules never change the state.
- `interface/` contains the functions that change the state (entry, exit, no-show, cancellation, waiting-list scan, clock advance) and the price calculation.
- `interface/engine.py` contains the class `Engine`. `Engine.execute(command)` dispatches a typed command with structural pattern matching, calls the matching rule function, applies the state change only if the rules returned `Ok`, and returns a typed result. The engine does not read or write files.

Architecture. Both entry points call the same `Engine`.

```
flowchart TB
  CLI["Command-line runner<br/>(start
file + JSONL commands)"] --> ENG
  GUI["Desktop application"] --> ENG
  ENG["Engine.execute(command)"] -->
RUL["rules/: all_rules_*<br/>read-only
validation"]
  ENG --> INT["interface/: state
changes<br/>clock, waiting list,
price"]
  RUL --> ATM["rules/atomic: shared
predicates"]
  RUL --> INT
  INT --> ST[("AppState<br/>config,
reservations, waiting list")]
  RUL -.reads.-> ST
```

Interaction with the desktop application. The command-line path parses each line of the command file into a typed command object (`CommandsFile`) and passes it to the engine. The desktop application does not read

a command file. When a start file is selected it builds a new `AppState` and a new `Engine`; every user action creates the same typed command object and calls `Engine.execute`; the returned result is displayed. A price preview uses `execute(PriceCommand, record=False)`, which does not change the state and is not logged. The engine keeps a history of `(command, result)` pairs, which is the data source for the event log. The application contains no business rule: even an invalid request is sent to the engine, which returns the error code. NOTE: TODO - confirm and adapt this paragraph after the GUI is finished.

2.2. Data Model

All entities are Pydantic models. Internal field names are English; the JSON field names of the specification (Turkish) are kept as validation and serialization aliases, so files are read and written in the required format. Timestamps are `datetime` objects serialized as `YYYY-MM-DDTHH:MM`.

TABLE 1. Entities of the data model (Turkish JSON names in parentheses)

Entity	Fields	Notes
ResourceType	`preparation_min` (hazirlik_dk), `base_fee` (temel_ucret), `energy_intensive` (enerji_yogun), `certificate` (sertifika)	Stored in a dictionary keyed by type name (ODA, LAB, YAZICI, CIHAZ).
Role	`weekly_quota` (haftalik_kota), `future_day` (ileri_gun), `max_duration_min` (azami_sure_dk)	Keyed by role name.
Resource	`id`, `name` (ad), `type` (tur), `allowed_roles` (izinli_roller)	Can be added at run time.
Maintenance Window	`resource` (kaynak), `start` (baslangic), `end` (bitis)	Not counted as usage; no preparation time after it.
User	`id`, `name` (ad), `role` (rol), `certificates` (sertifikalar), `balance` (bakiye), `reliability` (guvenilirlik)	Balance in KP; reliability 0-100.
Reservation	`id`, `user`, `resource`, `start`, `end`, `price` (ucret), `state` (durum), `package` (paket)	`id` is r1, r2, ... in creation order. State is one of ONAYLI, KULLANIMDA, TAMAMLANDI, IPTAL, GELMEDI.
WaitingEntry	`id`, `user`, `resource`, `start`, `end`, `registered_at` (kayit_saati)	`id` is b1, b2, ...
AppState	`config` (initial data and the simulated clock), `reservations`, `waiting_list`, `next_waiting_id`	The only mutable state; kept in memory.

The simulated clock is stored in `config.time`. Derived values (occupancy, price, waiting position) are computed when needed and are never stored, so no cache can become outdated. Reservation identifiers are generated from the number of reservations, which is valid because reservations are never deleted. Package identifiers are the largest existing package number plus one. Waiting identifiers use a separate counter (`next_waiting_id`), because waiting entries are removed when they are promoted or expire, and an identifier must never be reused.

2.3. Application of the Rules

Validation order (K10). A reservation request passes thirteen checks in a fixed order and the code of the first failing check is returned (Algorithm 1). All intervals are half-open, `[start, end)`. Two intervals a and b intersect if `a.start < b.end` and `b.start < a.end`. The preparation check widens the existing reservation by the preparation time h of the resource type on both sides, and the consecutive-use check does the same with 60 minutes for the same user and resource. The status of the existing reservation decides whether it is counted (Table 2). The price is computed only after all other checks have passed, because the balance check is the last one.

TABLE 2. Reservation states counted by each check

Check	Counted states
KESISME	ONAYLI, KULLANIMDA
HAZIRLIK_IHLALI, ARDISIK_KULLANIM	ONAYLI, KULLANIMDA, TAMAMLANDI
Occupancy for pricing	ONAYLI, KULLANIMDA (same resource type)
All checks	IPTAL and GELMEDI are never counted

Pricing (K5). The price is calculated for every 15-minute slice s of the request (Algorithm 2):

$$\text{price}=\operatorname{round}_{\uparrow}\Big(\sum_{s}\; B\cdot 0.25\cdot m_{band}(s)\cdot m_{peak}(s)\Big),\quad m_{band}=1+0.25\,\lfloor 4\rho \rfloor,\quad \rho=\frac{\text{occupied minutes in band}}{n_{type}\cdot 120}$$

where B is the base hourly fee of the type, n_{type} is the number of resources of that type at the moment of the request, and `round↑` rounds half up. The day is divided into seven two-hour bands (08-10, ..., 20-22). The occupancy of a band is the sum of the minutes of ONAYLI and KULLANIMDA reservations of the same type that lie inside the band; the request itself and maintenance windows are not counted. The peak multiplier is 1.5 for energy-intensive types between 10:00 and 16:00, otherwise 1.0. Python's built-in `round` uses banker's rounding, so the final rounding is done with `decimal.Decimal` and `ROUND_HALF_UP`. All multipliers are multiples of 0.25, so the floating-point sums are exact.

Balance and weekly quota (K6). The price is deducted when the reservation is accepted. When the clock passes a Monday 00:00, every balance is *set* to the weekly quota of the user's role; unused points are not carried over. A refund is limited so that the balance never exceeds the quota: `credited = min(quota, balance + refund) - balance`, and this credited value is the refund reported in the result.

State machine and reliability (K8). Table 3 summarizes the transitions. Entry is allowed from 10 minutes before to 15 minutes after the start time (otherwise GIRIS_PENCERESI_DISI). Entry, exit and cancellation check BULUNAMADI first and DURUM_HATASI second; entry then checks the time window. A reliability below 50 reduces the booking horizon to one day and forbids packages.

TABLE 3. Transitions, refunds and reliability changes

Event	Refund	Reliability
Cancel, 24 h or more before start	100%	unchanged
Cancel, from 2 h up to 24 h before start	50%, rounded down	unchanged

Event	Refund	Reliability
Cancel, less than 2 h before start	0	-5
No-show (no entry until start + 15 min)	0	-15 (minimum 0)
Completed at the end time	—	+2 (maximum 100)
Early exit (`cikis`)	none, end time := now	unchanged

Package reservations (K7). A package of two to four items is accepted completely or not at all. Checks run in this order: BULUNAMADI, PAKET_BOYUTU, GUVENILIRLIK_DUSUK, PAKET_RED, PAKET_ICI_KESISME, BAKIYE_YETERSIZ. Each item is validated separately with the K10 checks except the balance check, and the status of every item (TAMAM or the code) is returned in the result. Items are priced against the state before the package, so other items of the same package do not raise the occupancy. Conflicts between items of the same package (intersection, preparation time, consecutive use) are detected pairwise. The state is changed only after all checks have passed: the total is deducted once and all reservations get the same package identifier.

Waiting list (K9). A waiting entry is accepted only when the request fails because of KESISME or HAZIRLIK_IHLALI; any earlier failure returns its own code, and a request that is already valid returns BEKLEME_GEREKSIZ. The balance is not checked at this point. When a resource becomes free (cancellation, no-show or early exit) the list of that resource is scanned (Algorithm 4). Candidates are ordered by reliability (high first), then by registration time, then by the numeric part of the identifier (b2 before b10). Every candidate goes through the complete K10 validation with the current clock and balance; a candidate that passes becomes a reservation at the current price, the others stay in the list, and the scan continues to the end. Entries whose start time has passed are deleted. Automatic completion by the clock at the end time does not trigger a scan, because the time it frees is already in the past; an explicit exit does.

Simulated clock (K11). `advance_clock` collects all events in the interval (old time, new time], including an event exactly at the new time (Algorithm 3). Events are placed in a priority queue ordered by time, then by type priority (weekly reset, completion, no-show), then by the numeric reservation number. For each instant the clock is first moved to that instant, because the waiting-list scan uses the clock for timestamps and for the removal of expired entries. After the weekly resets, completions and no-shows of that instant, the waiting lists of the freed resources are scanned once per resource. A reservation created by an automatic assignment may itself become a no-show inside the same window, so its no-show is added to the queue immediately. A new time before the current time returns SAAT_GERI.

2.4. Pseudo-code and State Machine

Algorithm 1. Validation of one reservation request (K10)

```
function VALIDATE(u, r, b, e, S):
  if u or r unknown:
return BULUNAMADI
  if b mod 15 != 0 or e mod 15 != 0
     or e <= b:
return ZAMAN_ADIMI_HATASI
  if b < S.clock:
return GECMIS_ZAMAN
  if b < 08:00 or e > 22:00 (same
day):    return CALISMA_SAATI_DISI
  if e-b < 30 min or e-b >
role(u).max:    return SURE_SINIRI
```

```
  H <- 1 day if rel(u) < 50 else
role(u).future_day
  if b > S.clock + H:
return ILERI_TARIH_SINIRI
  if role(u) not in r.allowed_roles:
return ROL_IZNI_YOK
  if type(r) needs cert and u lacks
it:    return SERTIFIKA_YOK
  if overlaps maintenance window of r:
return BAKIM
  if overlaps ONAYLI/KULLANIMDA on r:
return KESISME
  if exists x in
{ONAYLI,KULLANIMDA,TAMAMLANDI} on r
with
        b < x.e + h and x.b < e + h:
return HAZIRLIK_IHLALI
  if same rule with +60 min, x.user ==
u:  return ARDISIK_KULLANIM
  p <- PRICE(r, b, e, S)
  if p > bal(u):
return BAKIYE_YETERSIZ
  return OK(p, bal(u) - p)
```

Algorithm 2. Price of a request (K5)

```
function PRICE(r, b, e, S):
  total <- 0
  for t in b, b+15min, ..., e-15min:
    band  <- [t - (t.hour mod 2)h -
t.minute, +2h)
    busy  <- sum of overlap(x, band)
for x in S.reservations
             with x.state in {ONAYLI,
KULLANIMDA}
             and type(x.resource) =
type(r)
    rho   <- busy / (count(resources
of type(r)) * 120)
    mband <- 1 + 0.25 * floor(4 * rho)
    mpeak <- 1.5 if
energy_intensive(type(r))
                    and 10:00 <= t <
16:00 else 1
    total <- total + base_fee * 0.25 *
mband * mpeak
  return ROUND_HALF_UP(total)
```

Algorithm 3. Advancing the simulated clock (K11)

```
function ADVANCE(S, T):
  Q <- min-heap ordered by (time,
priority, seq)
  push (Monday 00:00 in (S.clock, T],
0, weekly)
  for x in S.reservations:
    if x.state = KULLANIMDA and
S.clock < x.end <= T:
       push (x.end, 1, complete x)
    if x.state = ONAYLI and S.clock <
x.start+15 <= T:
       push (x.start+15, 2, no-show
x)    # ties: numeric id
  while Q not empty:
    t <- time of Q.top;  S.clock <- t;
freed <- []
    for each job at time t, in heap
order:
      weekly   : bal(u) <-
quota(role(u)) for all users
      complete : x.state <-
TAMAMLANDI; rel <- min(100, rel+2)
      no-show  : x.state <- GELMEDI;
rel <- max(0, rel-15)
                 freed.add(x.resource)
    for r in unique(freed): events +=
WAITSCAN(S, r)
    for each reservation y created in
this step:
      if y.start+15 <= T: push
(y.start+15, 2, no-show y)
  drop waiting entries with start < T;
S.clock <- T
```

Algorithm 4. Waiting-list scan for a freed resource (K9)

```
function WAITSCAN(S, r):
  drop waiting entries with start <
S.clock
  C <- entries of r sorted by (-
rel(user), registered_at, numeric id)
  for w in C:
    res <- VALIDATE(w.user, r,
w.start, w.end, S)   # full K10
    if res = OK(p, bal'):
      bal(w.user) <- bal';  create
ONAYLI reservation (price p)
```

```
      remove w;  emit OTOMATIK_ATAMA
  # entries that fail stay in the list
```

Reservation state machine.

```
stateDiagram-v2
  [*] --> ONAYLI : rezervasyon / paket
/ OTOMATIK_ATAMA
  ONAYLI --> KULLANIMDA : giris, from
start-10 to start+15
  ONAYLI --> GELMEDI : no entry until
start+15 (-15)
  ONAYLI --> IPTAL : iptal (refund by
lead time)
  KULLANIMDA --> TAMAMLANDI : end time
(+2) or cikis (early)
  GELMEDI --> [*]
  IPTAL --> [*]
  TAMAMLANDI --> [*]
```

3. Experimental Results

Test strategy. The suite consists of 16 test functions in 13 files, one file for each command family (`kullanici_ekle`, `kaynak_ekle`, `rezervasyon`, `fiyat`, `paket`, `bekleme`, `saat`, `giris`, `cikis`, `iptal`, `sorgu` for users and for resources) and one end-to-end scenario. Most tests are data driven. A folder `tests/data/<area>/` holds `initial_data.json`, a JSON Lines command file and a JSON Lines file with the expected results. The test builds the state, seeds reservations or waiting entries that are difficult to reach by commands, replays the commands and compares every produced result with the expected one as a typed object, so every field of the result must be equal to the expected value. The end-to-end test `test_full` replays a complete command file through `Engine.execute` and compares the whole result list. NOTE: TODO - paste the final `pytest` summary line here (number of passed tests) and confirm that `tests/data/full` is the course's 19-command sample scenario.

Edge cases checked by the tests. Table 4 lists the cases that were written to catch the most likely mistakes.

TABLE 4. Selected edge cases and the behavior that is verified

Situation	Verified behavior
A completion and a no-show at the same instant (11:00)	Completion is processed first.
A no-show exactly at Monday 00:00	The weekly reset runs before the no-show.
Reliability near the bounds	It never goes below 0 and never above 100.
An automatic assignment (r10) whose own no-show falls in the same clock window	The no-show is also processed in the same `saat` command.
A reservation completes at its end time while a waiting entry exists for the resource	No scan is triggered; the entry is not promoted and later expires.
Waiting entries b1 and b4 have equal reliability and registration time, and b4 is invalid (its end is not on a 15-minute step)	The smaller id (b1) is served first; the invalid entry b4 is skipped and the scan continues.
A waiting entry is removed and a new one is added	The new id is never a reused id (b4, not b3; the counter is also kept after removal).
Cancellation 1 h, 4 h and 26 h before start	Refund 0 and reliability -5; refund floor(15 x 0.5) = 7; refund 100% but limited by the quota, so 0 is credited.

Sample output. The following result lines are the expected lines of the course's sample scenario (the simulated clock starts on

Monday, 8 March 2027, 08:00) that the engine output is compared with. NOTE: TODO - replace them with the real output of your program after you run the sample scenario.

```
{"no":5,"komut":"rezervasyon","durum":
"TAMAM","rezervasyon_id":"r1","ucret":
36,"bakiye":84}
{"no":6,"komut":"rezervasyon","durum":
"TAMAM","rezervasyon_id":"r2","ucret":
45,"bakiye":75}
{"no":10,"komut":"rezervasyon","durum"
:"RED","kod":"HAZIRLIK_IHLALI"}
{"no":12,"komut":"paket","durum":"TAMA
M","paket_id":"p1","rezervasyonlar":["
r3","r4"],"ucretler":[48,8],"toplam":5
6,"bakiye":244}
{"no":15,"komut":"bekleme","durum":"TA
MAM","bekleme_id":"b1","sira":1}
{"no":18,"komut":"iptal","durum":"TAMA
M","iade":4,"bakiye":248,"guvenilirlik
":80,"olaylar":[]}
```

Command 5 has eight peak slices on an empty band: 8 x 3 x 1.0 x 1.5 = 36 KP. In command 6 the band 10-12 holds 60 of 240 occupied minutes, so the occupancy is exactly 0.25, the band multiplier is 1.25 and the price is 8 x 3 x 1.25 x 1.5 = 45 KP. Command 10 fails with HAZIRLIK_IHLALI because the preparation time of r1 ends at 13:20, and command 12 shows that package items are priced independently (48 and 8 KP).

4. Design Decisions

Language and data validation. Python 3.11 offers structural pattern matching, `StrEnum` and the `decimal` module, which fit the problem. Pydantic v2 validates every input at the boundary. Discriminated unions select the command class from the field `komut`, the event class from `tur` and the result class from `durum`, so a malformed line fails immediately. Aliases keep the Turkish JSON names of the specification while the code uses English names, and the same typed models serve as the test oracle.

Rules as read-only functions with a result type. A rule failure is an expected outcome, so rule functions return `Ok` or `Err` values instead of raising exceptions, and they never change the state. The engine changes the state only in the `Ok` branch, which makes packages and automatic assignments atomic. The same small predicates (`atomic_rule_reservation_*`) are shared by the reservation rules, the waiting rules and the waiting-list scan, so they cannot drift apart.

One engine without input or output. All behavior is reachable through `Engine.execute`. The command-line runner and the desktop application only translate between their own input and output and the typed commands and results, which guarantees that the screen shows the same results as the result file. The `record` flag allows a price preview without a log entry, and the history list supports the event log.

Clock and numbers. The clock first collects all jobs and then consumes them from a heap, so decisions are based on one consistent snapshot and the K11 order is a single sort key. Reservation numbers are compared as integers because text order would put r10 before r2. Prices and balances are integers; the pricing sum is exact for the allowed multipliers and is rounded with an explicit `ROUND_HALF_UP`.

Data structures. The state is kept in plain lists in memory, as the project allows, and searched linearly. For a campus-sized data set this is fast enough; the pricing cost grows with the number of slices times the number of reservations, and indexes by resource and day are a simple improvement if the data grows.

Desktop interface. NOTE: TODO - write after the GUI is implemented: the chosen UI library and why, the layout of the resource calendar (one row per resource, 08:00-22:00 axis, colors

by state, a separate sub-row for cancelled or no-show reservations that overlap a new reservation), the request form with a live price preview, the user panel, the action buttons placed at the reservation itself, the simulated-clock control and the event log. Relate each choice to the criteria: ease of operation, compactness, effortlessness, feedback and visual quality.

5. Conclusion

We implemented the reservation module of the Campus Resource Reservation System: the thirteen-step validation, maintenance, preparation and consecutive-use rules, occupancy-based pricing, the weekly quota, atomic packages, the state machine with refunds and reliability, the reliability-ordered waiting list with automatic assignment, and the deterministic simulated clock. All behavior is available through one engine without file input or output, and a data-driven test suite covers each command family and the difficult interactions between them.

The main difficulties were details that the rules leave to the implementer: the order of simultaneous events, an automatic assignment that creates a new event inside the window being processed, identifiers that must not be reused, integer ordering of identifiers, and exact half-up rounding. Each of them is covered by a dedicated test.

Future work. The command-line runner (`calistir.sh` and `calistir.bat`) and the desktop application remain to be completed and verified against the sample scenario (TODO: update this paragraph when they are finished). The package rules repeat the item checks of the single-reservation rules because each item must report its own status; they can be merged into one shared implementation. A circular import between the rules and the waiting-list functions (the waiting-identifier parser) should be moved to a neutral module. Indexes and an optional persistent store would allow larger campuses.

References

- [1] Kocaeli University, Dept. of Computer Engineering, "Yazilim Laboratuvari I, 2026-2027 Guz, Proje I: Kampus Kaynak Rezervasyon Sistemi, Rezervasyon Modulu ve Masaustu Uygulamasi," project specification, Oct. 2026.
- [2] Python Software Foundation, "Python 3.11 documentation," 2022. [Online]. Available: https://docs.python.org/3.11/
- [3] B. Bucher and G. van Rossum, "PEP 634 - Structural Pattern Matching: Specification," Python Enhancement Proposals, 2020. [Online]. Available: https://peps.python.org/pep-0634/
- [4] Pydantic Services Inc., "Pydantic documentation (v2)." [Online]. Available: https://docs.pydantic.dev/
- [5] Python Software Foundation, "heapq - Heap queue algorithm" and "decimal - Decimal fixed-point and floating-point arithmetic," Python 3.11 documentation. [Online]. Available: https://docs.python.org/3.11/library/
- [6] H. Krekel et al., "pytest documentation." [Online]. Available: https://docs.pytest.org/ (TODO: remove this reference if you do not run the tests with pytest.)
- [7] (TODO: add the documentation of the GUI library you choose.)